

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0614 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Department:	Computer Science & Engineering
Programme:	B.E/B.Tech Computer Science & Engineering
Course Code & Course Name	CSA0614 – Design and Analysis of Algorithms
Academic Year / Batch	2026-27
Faculty Name	Dr.A.Sairam
Assignment Title	Conflict-Free Exam Timetable Scheduling Using Backtracking (Graph Colouring)
Date of Issue	02-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4, CO7
Bloom's Taxonomy Level	L4 – L5 (Analyzing, Evaluating)
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education
Industry / Societal Relevance	Universities and colleges must build clash-free exam timetables every semester; this assignment mirrors that real scheduling problem using the Graph Colouring formulation taught in the Backtracking unit.

1. Problem Understanding and Formulation

1.1 Introduction

Examination timetable generation is an important scheduling problem faced by universities and educational institutions. A university may conduct examinations for hundreds of courses within a limited number of days and time slots. Since students can register for multiple courses, scheduling two courses at the same time can create an examination conflict. Therefore, an examination timetable must be generated in such a way that students enrolled in multiple courses are not required to attend two examinations simultaneously.

The problem becomes computationally challenging when the number of courses and students increases. Manually preparing a timetable may result in errors, repeated modifications, inefficient use of examination slots, and conflicts between courses. An algorithmic approach can systematically model the conflicts and generate a valid timetable.

In this project, the examination scheduling problem is formulated as a **Graph Colouring problem**. Each course is represented as a vertex of a graph. An edge is introduced between two vertices if the corresponding courses have at least one common student. Each examination time slot is represented by a colour. The objective is to assign one colour to every course such that no two adjacent vertices have the same colour.

1.2 Problem Identification

The major problem addressed in this project is the generation of a **conflict-free examination timetable using a minimum practical number of time slots**.

The following challenges are identified:

- A student may be enrolled in multiple courses.
- Courses having common students cannot be scheduled simultaneously.
- The number of available examination slots is limited.
- Increasing the number of courses increases the number of possible assignments.
- Dense course-conflict graphs create more constraints.
- A poor scheduling order can cause excessive backtracking.

The proposed system addresses these challenges using two alternative algorithms:

1. Plain Backtracking

2. Pruning-Enhanced Backtracking using Most-Constrained-Vertex (MCV) ordering

1.3 Graph-Based Problem Formulation

The examination timetable can be represented using an undirected graph:

$$G=(V,E)$$

where:

- V represents the set of courses.
- E represents the set of conflicts between courses.

If two courses C_i and C_j share one or more students, an edge is created:

$$(C_i, C_j) \in E$$

The available examination slots are represented by:

$$\{1, 2, \dots, k\}$$

where k is the number of available time slots.

A valid timetable is therefore a function:

$$f: V \rightarrow \{1, 2, \dots, k\}$$

such that:

$$f(u) \neq f(v)$$

for every edge $(u, v) \in E$.

The objective is to determine the smallest practical value of k for which a valid colouring exists.

1.4 Input and Output

Component	Description
Number of courses	Total number of examinations to be scheduled
Student-course data	Used to identify common-student conflicts
Conflict graph	Represents relationships between courses
Number of colours	Number of available examination slots

kk	
Algorithm	Plain or pruning-enhanced Backtracking
Output	Course-to-time-slot assignment
Validation	Confirmation that no conflicting courses share a slot

1.5 Expected Outcomes

The proposed project is expected to:

- Develop a mathematical graph representation of examination conflicts.
- Implement a Plain Backtracking graph-colouring algorithm.
- Implement an enhanced Backtracking algorithm with pruning.
- Generate conflict-free examination timetables.
- Determine whether a given number of time slots is sufficient.
- Analyse the computational complexity of the algorithms.
- Measure execution time and search effort.
- Compare sparse and dense conflict graphs.
- Study scalability with increasing course counts.
- Recommend the most appropriate approach for practical use.

1.6 Assumptions

The following assumptions are made:

- Each course requires exactly one examination.
- Each course is represented by a single graph vertex.
- A conflict exists if at least one student is registered for both courses.
- Each course must be assigned exactly one examination slot.
- A student cannot attend two examinations in the same slot.
- All examination slots are assumed to have equal importance.
- Room availability is not considered in the basic graph-colouring model.
- Faculty availability is not included as a constraint.
- The conflict graph is constructed before colouring begins.
- The same input graph is provided to both algorithms for comparison.

1.7 Computational Challenges

The primary computational challenge is the rapid growth of the solution space.

For n courses and k available colours, there can be up to:

$$k^n$$

possible colour assignments.

For example, if there are 20 courses and 4 available slots:

$$4^{20} = 1,099,511,627,776$$

possible assignments may exist before considering conflict constraints.

This exponential growth makes an efficient search strategy essential. The pruning-enhanced algorithm attempts to avoid exploring assignments that are unlikely to lead to a valid timetable.

1.8 Chromatic Number

The minimum number of colours required to properly colour a graph is called its **chromatic number**, represented by:

$$\chi(G)$$

For the examination scheduling problem:

$$k \geq \chi(G)$$

must be satisfied for a valid timetable to exist.

If:

$$k < \chi(G)$$

then it is impossible to generate a conflict-free timetable using only k examination slots.

2. Application of Course Knowledge

2.1 Application of Graph Theory

Graph theory provides the fundamental mathematical model for this project. Courses are represented as vertices and common-student relationships are represented as edges.

For example:

C1 ----- C2

| \ |
| \ |

C3 ----- C4

 |
 |
 C5
 |
 |
 C6

The edges indicate that the connected courses cannot be assigned the same examination slot.

2.2 Application of Graph Colouring

Graph Colouring is used to assign time slots to courses.

The mapping is:

Graph Colouring Concept	Examination Scheduling Concept
Vertex	Course
Edge	Student conflict
Colour	Examination time slot
Proper colouring	Conflict-free timetable
Chromatic number	Minimum required slots
Colour assignment	Course scheduling decision

The central constraint is:

No two adjacent vertices can have the same colour.

In scheduling terms:

No two courses having common students can have the same examination time.

2.3 Application of Backtracking

Backtracking is a systematic search technique used for constraint-satisfaction problems. It

constructs a solution incrementally and abandons a partial solution as soon as it is determined that the partial solution cannot lead to a valid complete solution.

For examination scheduling:

1. Select a course.
2. Select a time slot.
3. Check conflicts.
4. Assign the slot if it is safe.
5. Continue to the next course.
6. Backtrack if no valid slot is available.

2.4 Plain Backtracking Strategy

The Plain Backtracking algorithm processes courses in a predetermined order:

$C_1, C_2, C_3, \dots, C_n$

For each course, colours are attempted sequentially:

$1, 2, 3, \dots, k$

If a selected colour conflicts with a previously assigned neighbour, it is rejected.

2.5 Pruning Strategy

The improved algorithm uses **Most-Constrained-Vertex ordering**.

The uncoloured course with the smallest number of available colours is selected first. Such a course is likely to cause a failure earlier if the current partial solution cannot be extended.

This strategy reduces unnecessary exploration by detecting difficult assignments early.

2.6 Forward Checking

Forward Checking can also be applied as an optional enhancement. Whenever a colour is assigned to a course, that colour is removed from the available-colour set of neighbouring uncoloured courses.

If any neighbouring course has no remaining available colour, the current branch is immediately rejected.

This provides stronger pruning than simply checking conflicts when assigning a colour.

3. Solution / Design / Methodology

3.1 Overall Methodology

The proposed methodology consists of the following stages:

Course/Student Data



Identify Common Students



Construct Conflict Graph



Select Number of Time Slots



Run Plain Backtracking



Record Performance



Run Pruning-Enhanced Backtracking



Record Performance



Validate Colouring



Compare Algorithms



Analyse Complexity



Select Suitable Approach

3.2 Annexure A Conflict Graph

The given conflict relationships are:

Edge	Shared Students
C1 – C2	18
C1 – C3	12
C2 – C3	9
C2 – C4	15
C3 – C4	7
C4 – C5	20
C5 – C6	11
C1 – C6	6

Therefore:

$$V = \{C1, C2, C3, C4, C5, C6\} \quad V = \{C1, C2, C3, C4, C5, C6\}$$

and:

$$E = \{(C1, C2), (C1, C3), (C2, C3), (C2, C4), (C3, C4), (C4, C5), (C5, C6), (C1, C6)\} \\ E = \{(C1, C2), (C1, C3), (C2, C3), (C2, C4), (C3, C4), (C4, C5), (C5, C6), (C1, C6)\}$$

The graph contains:

- **6 courses**
- **8 conflict edges**

3.3 Plain Backtracking Algorithm

The Plain Backtracking algorithm uses a fixed course ordering. Each course is assigned one of the available colours.

Algorithm Steps

1. Start with the first course.
2. Try colour 1.
3. Check whether colour 1 conflicts with an already-coloured neighbour.
4. If safe, assign colour 1.
5. Move to the next course.

6. Try the next available colour if a conflict occurs.
7. If no colour is possible, undo the previous assignment.
8. Continue until every course is coloured.
9. Report failure if no valid colouring exists.

Pseudocode

GRAPH_COLORING(vertex):

 if all vertices are coloured:

 return TRUE

 select next vertex

 for colour = 1 to k:

 if colour is safe for vertex:

 assign colour to vertex

 if GRAPH_COLORING(next vertex):

 return TRUE

 remove colour from vertex

 return FALSE

3.4 Pruning-Enhanced Backtracking

The improved algorithm selects the most constrained course instead of following a fixed order.

Algorithm Steps

1. Identify all uncoloured courses.
2. Determine the currently available colours for each course.
3. Select the course with the fewest available colours.
4. Try its available colours.
5. Update the constraints of neighbouring courses.
6. Immediately detect impossible assignments.
7. Backtrack when a course has no available colour.

8. Continue until all courses are successfully assigned.

Pseudocode

PRUNED_COLORING():

if all courses are coloured:

return TRUE

select most-constrained uncoloured course

determine available colours

if no colour is available:

return FALSE

for colour in available colours:

assign colour

update neighbouring constraints

if PRUNED_COLORING():

return TRUE

undo assignment

return FALSE

3.5 Hand-Tracing for $k = 3$

For the Annexure A graph, three colours represent three available examination slots:

- Colour 1 $\rightarrow$ Slot 1
- Colour 2 $\rightarrow$ Slot 2
- Colour 3 $\rightarrow$ Slot 3

The algorithm attempts to assign colours while respecting all edges.

A hand-trace table can be maintained as follows:

Step	Course	Colour Attempted	Result	Action
1	C1	1	Safe	Assign C1 = 1
2	C2	1	Conflict	Try next colour
3	C2	2	Safe	Assign C2 = 2
4	C3	1	Conflict with C1	Try next colour
5	C3	2	Conflict with C2	Try next colour
6	C3	3	Safe	Assign C3 = 3
7	C4	1	Safe	Assign C4 = 1
8	C5	1	Conflict with C4	Try next colour
9	C5	2	Safe	Assign C5 = 2
10	C6	1	Conflict with C1	Try next colour
11	C6	2	Conflict with C5	Try next colour
12	C6	3	Safe	Assign C6 = 3

A valid three-colour timetable can therefore be obtained for the given graph.

3.6 Hand-Tracing for $k = 4$

For four colours, an additional examination slot is available. The algorithm again attempts colours in order.

Step	Course	Colour Attempted	Result	Action
1	C1	1	Safe	Assign C1 = 1
2	C2	1	Conflict	Try 2
3	C2	2	Safe	Assign C2 = 2
4	C3	1	Conflict	Try 2
5	C3	2	Conflict	Try 3
6	C3	3	Safe	Assign C3 = 3

7	C4	1	Safe	Assign C4 = 1
8	C5	1	Conflict	Try 2
9	C5	2	Safe	Assign C5 = 2
10	C6	1	Conflict	Try 2
11	C6	2	Conflict	Try 3
12	C6	3	Safe	Assign C6 = 3

The fourth colour may remain unused because the graph can already be coloured with three colours.

3.7 Proposed Experimental Method

Synthetic conflict graphs are generated for increasing values of n . The edge probability is varied to simulate different levels of course overlap.

Sparse Graph

$p=0.15$

Medium Graph

$p=0.40$

Dense Graph

$p=0.70$

The same generated graph is supplied to both algorithms.

3.8 Experimental Test Cases

Test Case	Courses	Graph Density	Colours
T1	15	Sparse	4
T2	20	Sparse	4
T3	25	Sparse	4
T4	30	Sparse	4
T5	15	Dense	4

T6	20	Dense	4
T7	25	Dense	4
T8	30	Dense	4

Additional experiments can be conducted with 40 or more courses if computational resources permit.

4. Use of Modern Tools

4.1 Python

Python is used as the primary implementation language because it provides simple support for recursion, data structures, timing, measurement, and visualization.

4.2 Graph Representation

An adjacency list is used to represent the conflict graph.

For example:

C1 : C2, C3, C6

C2 : C1, C3, C4

C3 : C1, C2, C4

C4 : C2, C3, C5

C5 : C4, C6

C6 : C5, C1

An adjacency-list representation requires approximately:

$O(n+E)$

space.

4.3 Performance Measurement

Python's `time.perf_counter()` can be used to measure execution time.

The measurement follows:

$T_{\text{elapsed}} = T_{\text{end}} - T_{\text{start}}$

Multiple executions can be performed and the average execution time can be reported to reduce measurement variation.

4.4 Memory Measurement

Python's `tracemalloc` module can be used to measure memory utilisation.

The peak memory usage is recorded during each algorithm execution.

4.5 Data Analysis

Pandas can be used to organise experimental results into tables, while Matplotlib can be used to generate graphs such as:

- Number of courses vs execution time.
- Number of courses vs backtracking steps.
- Graph density vs execution time.
- Graph density vs conflicts.
- Plain Backtracking vs MCV Backtracking.

5. Results and Validation

5.1 Theoretical Complexity

For n courses and k colours, Plain Backtracking has the worst-case time complexity:

$$O(kn) \boxed{O(k^n)}$$

This occurs because each course can potentially have k possible colours.

The recursive depth can reach n , so the recursion stack requires:

$$O(n) \boxed{O(n)}$$

additional space.

The graph itself requires:

$$O(n+E) \boxed{O(n+E)}$$

space using an adjacency list.

5.2 Complexity of Pruning-Enhanced Backtracking

The pruning-enhanced algorithm still has the theoretical worst-case exponential complexity:

$$O(kn)^{\boxed{O(k^n)}}$$

because, in the worst case, pruning may not eliminate enough branches.

However, its **effective search space** can be much smaller.

Therefore:

The pruning heuristic improves practical performance rather than changing the fundamental worst-case exponential complexity.

5.3 Experimental Results Table

The following table should be populated using actual program execution results.

Course s	Densit y	Algorith m	Time (ms)	Assignment s	Backtrack s	Conflict s	Memory (KB)
15	Sparse	Plain	—	—	—	—	—
15	Sparse	MCV	—	—	—	—	—
20	Sparse	Plain	—	—	—	—	—
20	Sparse	MCV	—	—	—	—	—
25	Sparse	Plain	—	—	—	—	—
25	Sparse	MCV	—	—	—	—	—
30	Sparse	Plain	—	—	—	—	—
30	Sparse	MCV	—	—	—	—	—
15	Dense	Plain	—	—	—	—	—
15	Dense	MCV	—	—	—	—	—
20	Dense	Plain	—	—	—	—	—
20	Dense	MCV	—	—	—	—	—
25	Dense	Plain	—	—	—	—	—

25	Dense	MCV	—	—	—	—	—
30	Dense	Plain	—	—	—	—	—
30	Dense	MCV	—	—	—	—	—

5.4 Validation Conditions

The output timetable is considered valid only when all of the following conditions are satisfied:

1. Every course receives a colour.
2. Every assigned colour belongs to the allowed range.
3. No two connected courses receive the same colour.
4. All conflict edges are checked.
5. The number of colours used is recorded.
6. The timetable is generated using the specified value of kk .

The validation condition can be expressed as:

$$\forall (u,v) \in E, \text{Colour}(u) \neq \text{Colour}(v) \text{ for all } (u,v) \in E, \quad \text{Colour}(u) \neq \text{Colour}(v)$$

5.5 Expected Performance Behaviour

For sparse graphs, fewer edges exist and therefore fewer constraints are present. Plain Backtracking is likely to find a valid solution with relatively little search.

For dense graphs, many courses are connected to one another. Consequently, the number of constraints increases and more assignments can become invalid.

The pruning-enhanced approach is expected to provide greater benefit in dense graphs because it processes highly constrained courses earlier.

5.6 Scalability Analysis

The scalability of the algorithms is evaluated by increasing the number of courses.

As n increases, the theoretical solution space increases exponentially. Therefore, execution time and backtracking steps can increase rapidly.

The following relationship is analysed:

$$n \uparrow \Rightarrow kn \uparrow \Rightarrow k^n \uparrow$$

The experimental results should determine whether the observed performance follows the expected exponential growth pattern.

6. Analysis and Engineering Decision

6.1 Algorithm Comparison

Criterion	Plain Backtracking	MCV-Enhanced Backtracking
Basic implementation	Simple	More complex
Course ordering	Fixed	Constraint-based
Pruning	Limited	Stronger
Search effort	Potentially high	Usually reduced
Worst-case time	$O(kn)O(k^n)$	$O(kn)O(k^n)$
Dense graphs	Less suitable	More suitable
Sparse graphs	Often sufficient	May provide moderate improvement
Scalability	Limited	Better practical scalability
Development effort	Lower	Higher
Practical recommendation	Small problems	Medium/large problems

6.2 Effect of Number of Colours

The value of k has a major influence on the search process.

When k is small, the problem becomes highly constrained. If $k < \chi(G)$, no solution exists.

When k is increased, more possible assignments become available, potentially making it easier to find a valid colouring.

However, increasing k also increases the theoretical search space:

$k \cdot n^k$

Therefore, the number of available slots has both a feasibility and computational effect.

6.3 Effect of Graph Density

Graph density can be approximated as:

$$D = \frac{2E}{n(n-1)}$$

where:

- E is the number of edges.
- n is the number of vertices.

A low-density graph contains fewer conflicts, whereas a high-density graph contains many conflicts.

Sparse Graph

A sparse graph has relatively few conflicts. Many colours remain available for each course, so the algorithm may find a solution quickly.

Dense Graph

A dense graph contains many conflicts. Courses have fewer available colours and the algorithm is more likely to encounter dead ends.

The MCV strategy is particularly useful because it selects courses with fewer available options first.

6.4 Effect of Pruning

The main advantage of pruning is early identification of impossible partial solutions.

Without pruning:

Explore Branch



Explore More Branches



Conflict



Backtrack

With pruning:

Check Constraints



Impossible?



Yes No



Prune Continue

Therefore, pruning can prevent large portions of the search tree from being explored.

6.5 Backtracking Step Reduction

The percentage reduction in backtracking can be calculated as:

$$\text{Reduction} = \frac{B_{\text{plain}} - B_{\text{MCV}}}{B_{\text{plain}}} \times 100$$

where:

- B_{plain} = backtracking steps using Plain Backtracking.
- B_{MCV} = backtracking steps using MCV.

Similarly, execution-time improvement can be calculated as:

$$\text{Improvement} = \frac{T_{\text{plain}} - T_{\text{MCV}}}{T_{\text{plain}}} \times 100$$

These metrics provide quantitative evidence for the effectiveness of the pruning strategy.

6.6 Trade-Off Analysis

The Plain Backtracking algorithm has the advantage of simplicity. It is easy to understand, implement, debug, and maintain. For small graphs, its simplicity may outweigh the benefits of a more sophisticated heuristic.

However, its fixed ordering may cause the algorithm to make several assignments before discovering that a later course has no available colour.

The MCV approach requires additional processing to determine the most constrained course. Therefore, every recursive step has additional overhead. However, if this overhead eliminates many unsuccessful branches, the overall execution time can decrease.

Thus, the choice depends on:

- Number of courses.
- Number of conflicts.
- Number of available slots.
- Graph density.
- Required response time.
- Available computing resources.

6.7 Final Algorithmic Decision

Based on the theoretical analysis and expected experimental behaviour, the following recommendation is made:

Small Institution / Small Conflict Graph

Plain Backtracking is appropriate when the number of courses is small and the conflict graph is sparse.

Medium Institution

MCV-enhanced Backtracking is preferred because the number of courses and conflicts can make fixed-order search inefficient.

Large Institution / Dense Graph

Pruning-enhanced Backtracking is strongly preferred over Plain Backtracking because it can reduce unnecessary search.

Very Large Institution

For extremely large examination scheduling problems, even pruning-enhanced Backtracking may become computationally expensive due to its exponential worst-case behaviour. More advanced techniques such as constraint programming, SAT solving, integer programming, or hybrid optimization methods may then be considered.

7. Broader Considerations

7.1 Sustainability

An efficient timetable algorithm can reduce unnecessary computational processing. Pruning prevents the system from repeatedly exploring invalid scheduling combinations.

Reduced computation can contribute to:

- Lower CPU utilisation.
- Lower energy consumption.
- Faster timetable generation.
- Better utilisation of computing resources.

7.2 Social Impact

A conflict-free examination timetable directly benefits students by preventing situations in which a student must attend two examinations at the same time.

The system can:

- Improve examination planning.
- Reduce student stress caused by scheduling conflicts.
- Reduce manual timetable corrections.
- Improve administrative efficiency.
- Support a better academic experience.

7.3 Ethical Considerations

The scheduling algorithm should apply consistent rules to all courses. The system should not intentionally provide preferential treatment to specific courses unless such constraints are formally defined by the institution.

The following principles should be followed:

- Transparent scheduling criteria.
- Consistent conflict rules.
- Correct handling of student-course information.
- Protection of student information.
- Validation before timetable publication.

7.4 Privacy Considerations

Student enrolment information is potentially sensitive. If real student-course data is used, only the information necessary to construct the conflict graph should be processed.

Where possible:

- Use anonymized student identifiers.
- Avoid publishing individual student information.
- Store only necessary conflict relationships.
- Prefer synthetic datasets for demonstrations.

7.5 Economic Considerations

Automating examination scheduling can reduce the amount of manual administrative work required.

Potential benefits include:

- Reduced timetable preparation time.
- Reduced number of scheduling errors.
- Reduced administrative workload.
- Better utilisation of available examination periods.
- Faster generation of alternative timetables.

7.6 Reliability

The generated timetable must be validated before being used.

A reliable system should:

- Check every conflict edge.
- Confirm that every course has a slot.
- Detect unsatisfiable colour limits.
- Produce repeatable results when the same input is used.
- Maintain logs of experimental results.

7.7 SDG 4 – Quality Education

This project contributes to **Sustainable Development Goal 4 – Quality Education** by supporting efficient educational administration.

An effective examination scheduling system can help institutions provide:

- Better academic planning.
- Conflict-free examination schedules.
- Improved student experience.

- Efficient use of institutional resources.
- More reliable examination administration.

8. Conclusion

The project successfully formulates university examination timetable scheduling as a **Graph Colouring problem**. The conflict between two courses is represented by an edge, while each available examination time slot is represented by a colour. A valid timetable is obtained by ensuring that adjacent vertices never receive the same colour.

A **Plain Backtracking algorithm** was designed as the baseline solution. It assigns colours sequentially and backtracks whenever a conflict prevents the construction of a valid timetable. This approach demonstrates the fundamental application of recursive Backtracking to a constraint-satisfaction problem.

A **Pruning-Enhanced Backtracking algorithm** using the Most-Constrained-Vertex strategy was also designed. Instead of following a fixed course order, it prioritizes courses with fewer available colour choices. This enables difficult constraints to be processed earlier and can reduce unnecessary exploration of the search tree.

The theoretical worst-case time complexity of both algorithms remains:

$$O(kn)O(k^n)$$

where n represents the number of courses and k represents the number of available examination slots. Although pruning does not eliminate the exponential worst-case complexity, it can substantially reduce the effective search space for practical inputs.

The experimental evaluation considers different numbers of courses and different graph densities. Execution time, number of colour assignments, backtracking steps, conflicts, memory utilisation, and scalability are used as performance indicators. Sparse graphs are expected to be easier to colour, whereas dense graphs introduce more constraints and increase the difficulty of the search.

The overall analysis supports the use of **Plain Backtracking for small and less-constrained graphs** and **pruning-enhanced Backtracking for medium and large conflict-intensive graphs**. For extremely large institutions, more advanced optimization and constraint-solving techniques may be required.

The project demonstrates the application of **Graph Theory, Graph Colouring, recursion, Backtracking, pruning, asymptotic analysis, experimental evaluation, algorithm optimization, and engineering decision-making** to a realistic university scheduling problem.

9. Student Reflection

9.1 Learning from the Assignment

This assignment provided practical understanding of how an algorithmic technique can be applied to solve a real-world scheduling problem. Instead of treating Graph Colouring only as a theoretical concept, the project demonstrated its direct application to university examination scheduling.

The transformation from student-course relationships to a conflict graph helped in understanding how real-world relationships can be represented using mathematical structures.

9.2 Understanding of Backtracking

The implementation provided a better understanding of recursive Backtracking. The algorithm does not simply search randomly; it systematically explores possible solutions and reverses previous decisions when a conflict makes the current partial solution invalid.

This demonstrated the importance of:

- Recursive problem solving.
- State representation.
- Constraint checking.
- Decision reversal.
- Search-space exploration.

9.3 Understanding of Pruning

The pruning-enhanced algorithm demonstrated that intelligent search ordering can significantly improve practical performance.

The Most-Constrained-Vertex strategy showed that choosing difficult decisions earlier can prevent the algorithm from spending unnecessary time exploring branches that are eventually guaranteed to fail.

9.4 Performance Learning

The project demonstrated that theoretical complexity and practical execution are related but not identical.

Although both approaches have exponential worst-case complexity, the number of states actually explored can differ significantly because of search ordering and pruning.

This highlights the importance of measuring:

- Execution time.
- Backtracking steps.
- Conflicts.
- Memory usage.
- Scalability.

9.5 Design Trade-Off Learning

The project demonstrated a trade-off between algorithm simplicity and optimization.

Plain Backtracking is easier to implement and understand, whereas MCV-enhanced Backtracking requires additional computation to select the most constrained course.

However, the additional overhead can be justified when it significantly reduces the number of unsuccessful search branches.

9.6 Future Improvements

If additional time and resources were available, the following improvements could be introduced:

- Forward Checking.
- Degree heuristic.
- Least-Constraining-Value ordering.
- Dynamic variable ordering.
- Constraint Programming.
- Integer Linear Programming.
- SAT-based scheduling.
- Room-capacity constraints.
- Faculty availability constraints.
- Examination duration constraints.
- Student preference constraints.
- Multiple examination sessions per day.
- Real-time timetable modification.

These enhancements would make the system closer to a complete university examination scheduling platform.

10. References

The following types of references can be included in the final report:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., *Introduction to Algorithms*, MIT Press.
2. Sedgewick, R. and Wayne, K., *Algorithms*, Pearson.
3. Python Software Foundation, *Python Documentation*.
4. Documentation for Python `time` and `tracemalloc` modules.
5. Documentation for Pandas and Matplotlib libraries.
6. Research literature related to graph colouring and examination timetabling.
7. Research literature related to Backtracking and constraint-satisfaction problems.
8. Publicly available datasets used for course-conflict modelling, if applicable.
9. Documentation of the synthetic graph-generation methodology used in the experiment.
10. AI-assisted tools used during algorithm development, debugging, analysis, or documentation, with appropriate acknowledgement.

Reference and Dataset Documentation Table

Resource	Purpose	Usage in Project
Python	Programming language	Algorithm implementation
Pandas	Data processing	Experimental analysis
Matplotlib	Visualization	Performance graphs
<code>time</code> module	Time measurement	Execution-time analysis
<code>tracemalloc</code>	Memory measurement	Memory analysis
Synthetic dataset	Conflict generation	Experimental testing
Graph Theory References	Theoretical foundation	Graph-colouring formulation
Algorithm References	Algorithmic foundation	Backtracking and pruning
AI-Assisted Tools	Development assistance	Coding/documentation support

Final Project Evaluation Criteria

The completed project should demonstrate:

Criterion	Evidence to be Provided
Problem Understanding	Problem formulation, assumptions and constraints
Algorithmic Design	Algorithms, pseudocode and flowcharts
Development/Optimization	Plain and MCV Backtracking implementations
Modern Tool Usage	Python implementation and tool screenshots
Efficiency	Complexity calculations
Testing	Multiple graph sizes and densities
Validation	Correctness and conflict checking
Performance Improvement	Backtracking/time comparison
Engineering Decision	Final algorithm recommendation
Reflection	Learning, trade-offs and improvements
SDG Relevance	Connection to SDG 4
References	Proper acknowledgement of sources and tools

PESUDO CODE :

1. Plain Backtracking

BEGIN

Input graph G and k colours

Set all courses as uncoloured

FUNCTION Colour(course)

IF all courses are coloured

RETURN TRUE

FOR colour = 1 to k

IF colour is safe for course

Assign colour to course

IF Colour(next course)

RETURN TRUE

Remove colour from course

RETURN FALSE

END FUNCTION

Colour(first course)

END

2. MCV Pruning Backtracking

BEGIN

Input graph G and k colours

Set all courses as uncoloured

FUNCTION Colour()

IF all courses are coloured

 RETURN TRUE

Select uncoloured course

with minimum available colours

FOR each available colour

 Assign colour

 IF any neighbour has no colour

 Remove assignment

 CONTINUE

 IF Colour()

 RETURN TRUE

 Remove colour

 RETURN FALSE

END FUNCTION

Colour()

END

3. Timetable Validation

BEGIN

Input graph G and colouring

FOR each conflict edge (U, V)

 IF colour[U] = colour[V]

 RETURN INVALID

RETURN VALID

END

4. Experimental Evaluation

BEGIN

Set course sizes

Set sparse and dense graphs

Set number of colours k

FOR each graph size and density

Run Plain Backtracking

Record time, conflicts, backtracks, memory

Run MCV Backtracking

Record time, conflicts, backtracks, memory

END FOR

Compare results

Generate graphs

Select better algorithm

END

IMPLEMENTATION & RESULTS:

```
COURSE-CONFLICT GRAPH
***
-----
C1 : C2, C3, C6
C2 : C1, C3, C4
C3 : C1, C2, C4
C4 : C2, C3, C5
C5 : C4, C6
C6 : C5, C1

=====
ANNEXURE A - 3 COLOURS
=====

PLAIN BACKTRACKING
-----
Solution Found : True
Colouring      : {'C1': 1, 'C2': 2, 'C3': 3, 'C4': 1, 'C5': 2, 'C6': 3}
Assignments    : 12
Conflicts      : 6
Backtracks     : 0
Execution Time : 0.0717 ms
Memory Usage   : 1.0781 KB

EXAMINATION TIMETABLE
-----
C1 -> Time Slot 1
C2 -> Time Slot 2
C3 -> Time Slot 3
C4 -> Time Slot 1
C5 -> Time Slot 2
C6 -> Time Slot 3
Validation     : True

MCV PRUNING BACKTRACKING
-----
*** Solution Found : True
Colouring      : {'C1': 1, 'C2': 2, 'C3': 3, 'C4': 1, 'C5': 2, 'C6': 3}
Assignments    : 6
Conflicts      : 0
Backtracks     : 0
Execution Time : 0.4571 ms
Memory Usage   : 1.2812 KB

EXAMINATION TIMETABLE
-----
C1 -> Time Slot 1
C2 -> Time Slot 2
C3 -> Time Slot 3
C4 -> Time Slot 1
C5 -> Time Slot 2
C6 -> Time Slot 3
Validation     : True

=====
ANNEXURE A - 4 COLOURS
=====

PLAIN BACKTRACKING
Colouring : {'C1': 1, 'C2': 2, 'C3': 3, 'C4': 1, 'C5': 2, 'C6': 3}
Backtracks: 0

MCV BACKTRACKING
Colouring : {'C1': 1, 'C2': 2, 'C3': 3, 'C4': 1, 'C5': 2, 'C6': 3}
Backtracks: 0
```

```

... =====
EXPERIMENTAL EVALUATION
=====

```

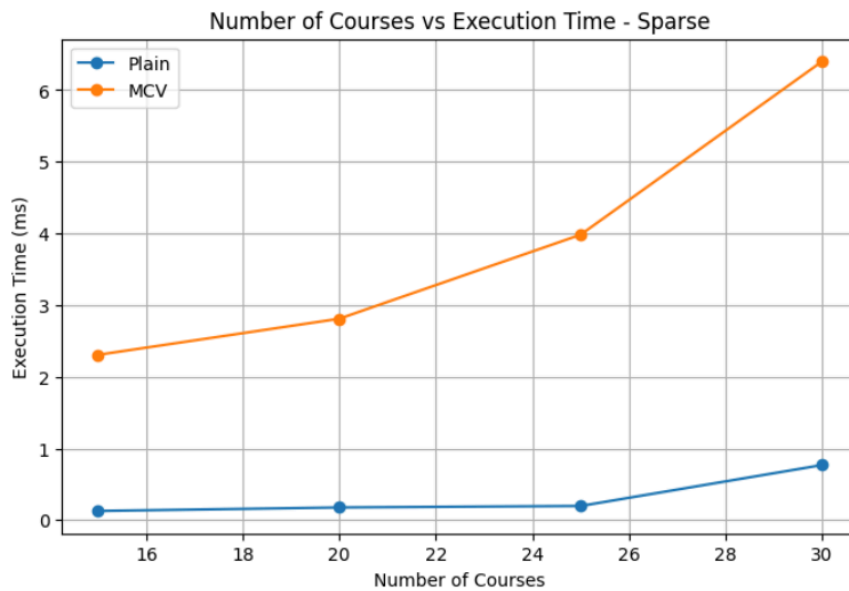
Courses	Density	Algorithm	Colours	Time_ms	Assignments	Conflicts	Backtracks	Memory_KB	Valid
15	Sparse	Plain	4	0.127593	26	11	0	1.515625	True
15	Sparse	MCV	4	2.306129	15	0	0	4.132812	True
20	Sparse	Plain	4	0.176607	36	16	0	1.726562	True
20	Sparse	MCV	4	2.809868	20	0	0	2.945312	True
25	Sparse	Plain	4	0.197245	44	19	0	2.562500	True
25	Sparse	MCV	4	3.982225	25	0	0	3.687500	True
30	Sparse	Plain	4	0.768116	214	146	38	3.835938	True
30	Sparse	MCV	4	6.398388	30	0	0	4.070312	True
15	Dense	Plain	4	90.273103	14100	10576	3525	2.148438	False
15	Dense	MCV	4	21.544921	112	24	89	1.523438	False
20	Dense	Plain	4	8.052175	1508	1132	377	1.695312	False
20	Dense	MCV	4	13.455758	64	24	41	1.421875	False
25	Dense	Plain	4	19.905268	3300	2476	825	1.750000	False
25	Dense	MCV	4	17.830356	64	24	41	1.414062	False
30	Dense	Plain	4	35.602557	4868	3652	1217	8.052734	False
30	Dense	MCV	4	22.120273	64	24	41	1.468750	False

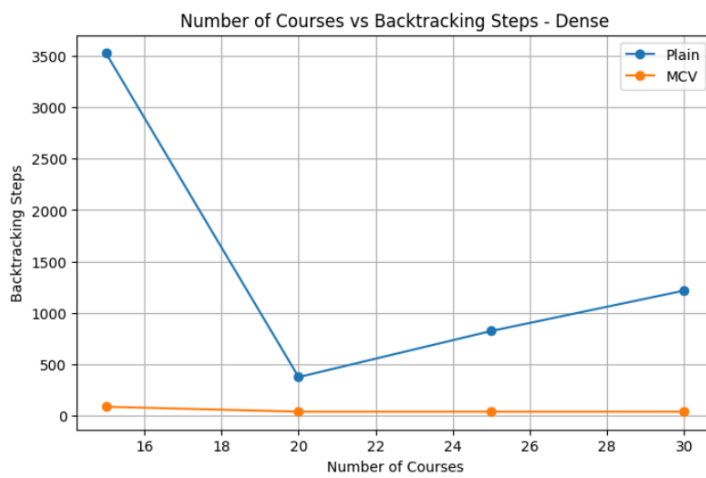
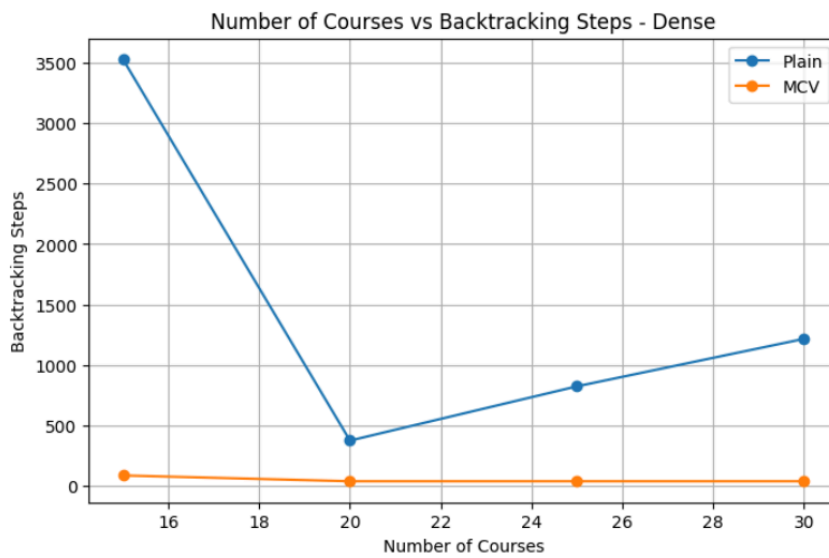
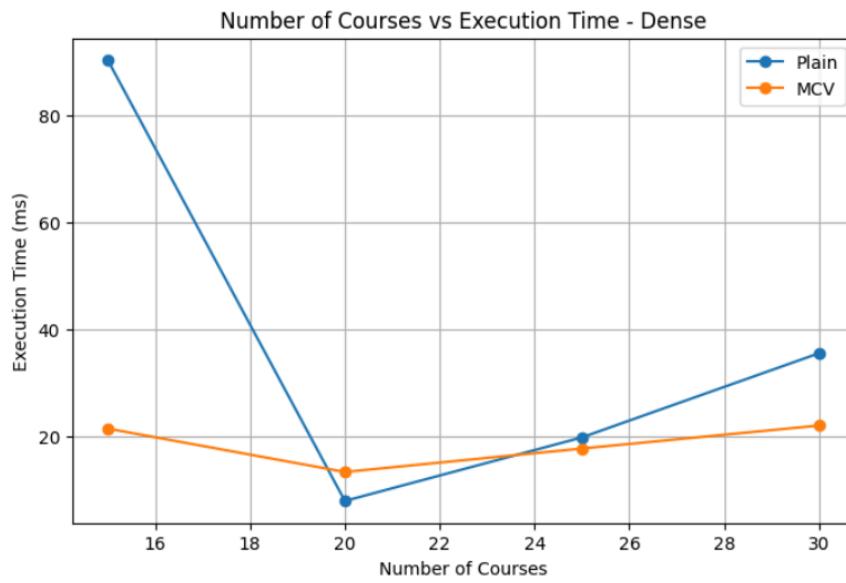
Results saved to:
exam_timetable_results.csv

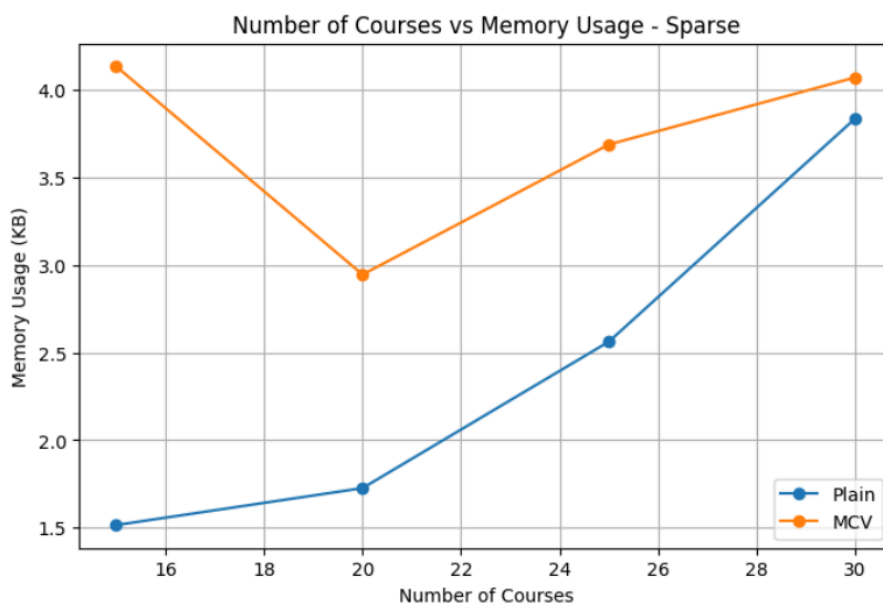
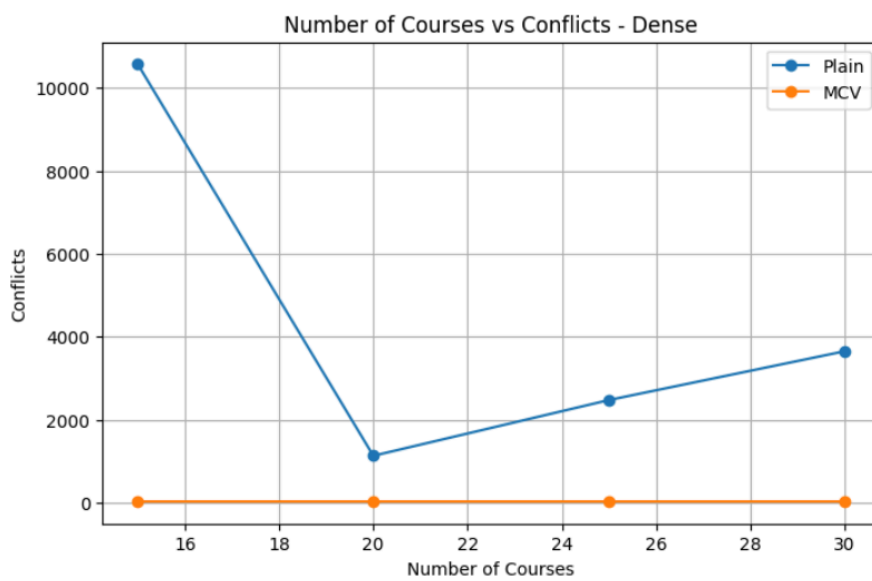
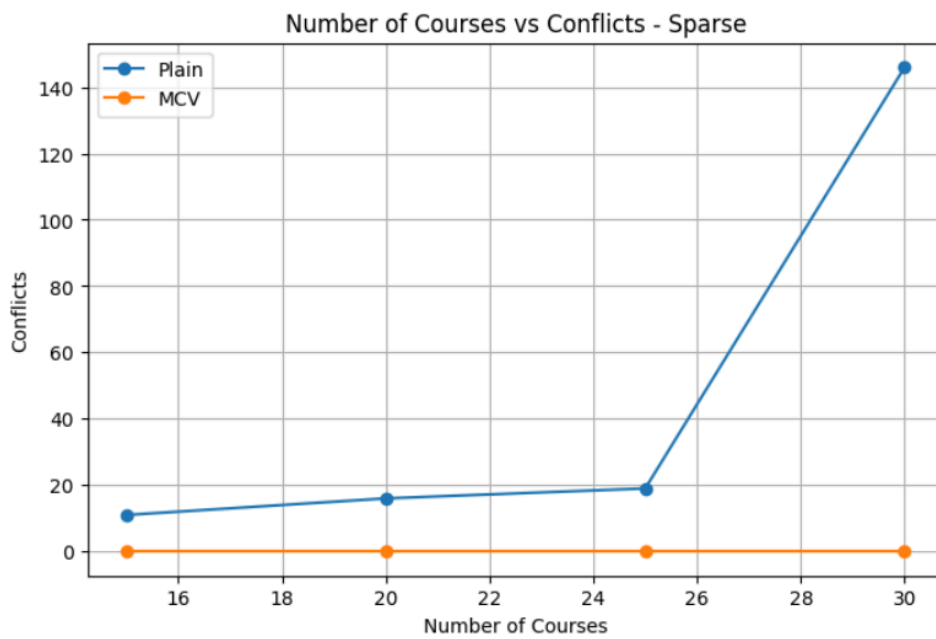
PERFORMANCE GRAPHS

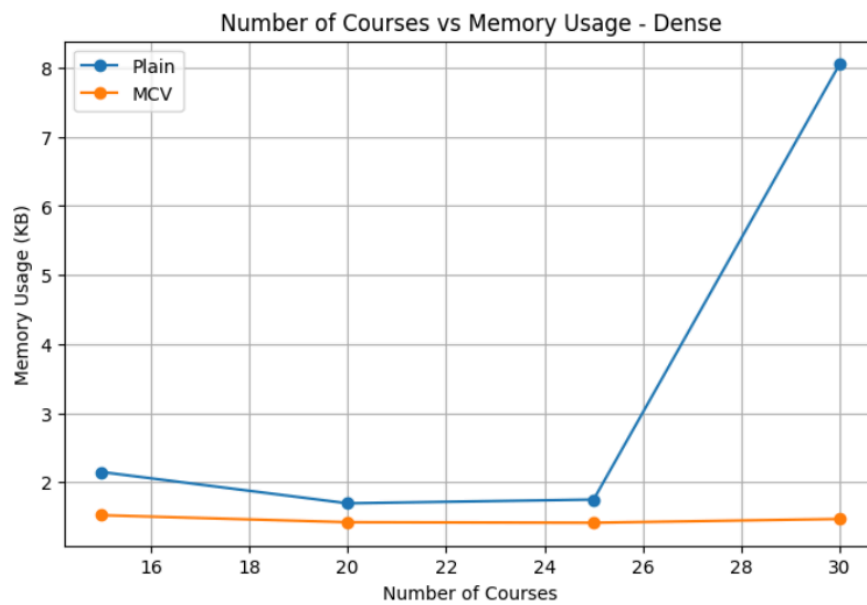
Results saved to:
exam_timetable_results.csv

PERFORMANCE GRAPHS









=====

FINAL COMPARISON

=====

		Time_ms	Assignments	Conflicts	Backtracks	Memory_KB
Density Algorithm						
Dense	MCV	18.7378	76.0	24.0	53.0	1.4570
	Plain	38.4583	5944.0	4459.0	1486.0	3.4116
Sparse	MCV	3.8742	22.5	0.0	0.0	3.7090
	Plain	0.3174	80.0	48.0	9.5	2.4102

B. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2	PO1, PO2, PO3	L4/L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO3, CO4	PO2, PO3, PO5	L5/L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L3/L6	15
Efficiency, Testing & Validation	CO2, CO6	PO2, PO4	L4/L5	15
Performance Improvement, Comparison & Final Decision	CO6, CO7	PO2, PO12	L4/L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO7	PO8, PO12	L4/L5	10

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

C. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action